11. In Shop X, there are 20 bottles of pure oil and 30 bottles of adulterated oil, while in Shop Y, there are 40 bottles of pure oil and 20 bottles of adulterated oil. A bottle is selected at random from one of the shops, with Shop Y being twice as likely to be chosen as Shop X. The selected bottle is found to be adulterated.

CODE:

```
from fractions import Fraction
# Prior probabilities: Y is twice as likely to be chosen as X
P_X = Fraction(1, 3)
P_Y = Fraction(2, 3)
# Likelihoods: P(adulterated | shop)
P_A_given_X = Fraction(30, 50)
P_A_given_Y = Fraction(20, 60)
# Total probability of picking an adulterated bottle (Law of Total Probability)
P_A = P_X * P_A_given_X + P_Y * P_A_given_Y
# Posterior probabilities via Bayes' theorem
P_X_given_A = (P_X * P_A_given_X) / P_A
P_Y_given_A = (P_Y * P_A_given_Y) / P_A
print(f"P(Shop X chosen) = {P_X} = {float(P_X):.4f}")
print(f"P(Shop Y chosen) = {P_Y} = {float(P_Y):.4f}")
print(f"P(Adulterated | X) = {P_A_given_X} = {float(P_A_given_X):.4f}")
print(f"P(Adulterated | Y) = {P_A_given_Y} = {float(P_A_given_Y):.4f}"
print(f"P(Adulterated) overall = {P_A} = {float(P_A):.4f}")
print()
print(f"P(Shop X | Adulterated) = {P_X_given_A} = {float(P_X_given_A):.4f}")
print(f"P(Shop Y | Adulterated) = {P_Y_given_A} ={float(P_Y_given_A):.4f}")
```

OUTPUT :

```
from fractions import Fraction
P_X = Fraction(1, 3)
P_Y = Fraction(2, 3)
P_A_given_X = Fraction(30, 50)
P_A_given_Y = Fraction(20, 60)
P_A = P_X * P_A_given_X + P_Y * P_A_given_Y
P_X_given_A = (P_X * P_A_given_X) / P_A
P_Y_given_A = (P_Y * P_A_given_Y) / P_A
print(f"P(Shop X chosen) = {P_X} = {float(P_X):.4f}")
print(f"P(Shop Y chosen) = {P_Y} = {float(P_Y):.4f}")
print(f"P(Adulterated | X) = {P_A_given_X} = {float(P_A_given_X):.4f}")
print(f"P(Adulterated | Y) = {P_A_given_Y} = {float(P_A_given_Y):.4f}")
print(f"P(Adulterated) overall = {P_A} = {float(P_A):.4f}")
print()
print(f"P(Shop X | Adulterated) = {P_X_given_A} = {float(P_X_given_A):.4f}")
print(f"P(Shop Y | Adulterated) = {P_Y_given_A} = {float(P_Y_given_A):.4f}")
```

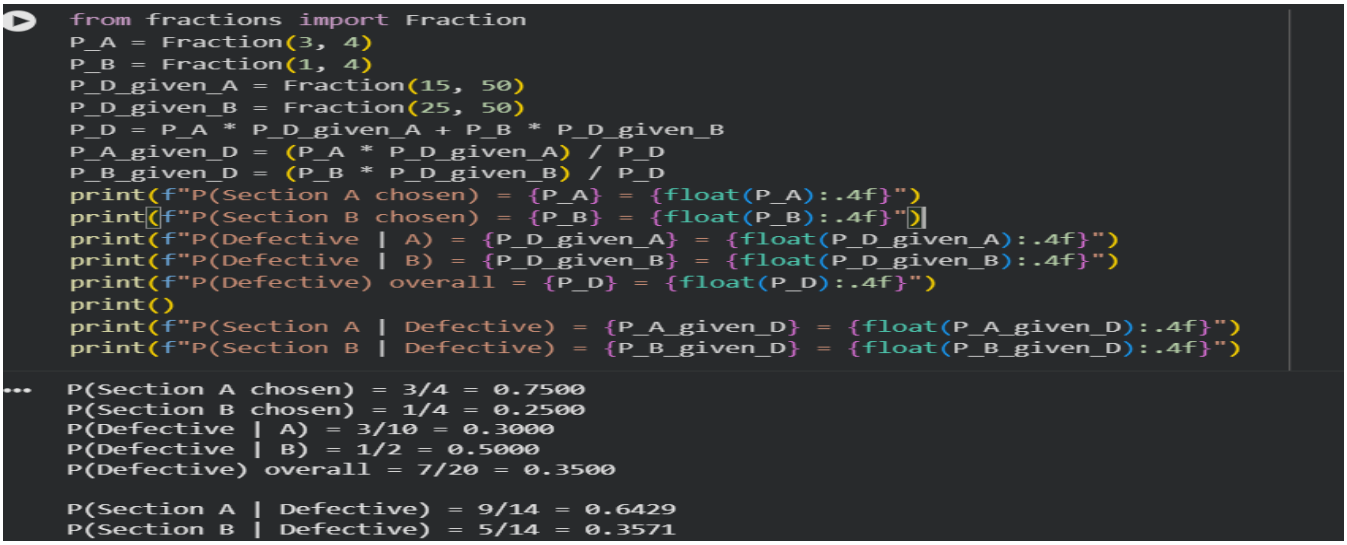
```
P(Shop X chosen) = 1/3 = 0.3333
P(Shop Y chosen) = 2/3 = 0.6667
P(Adulterated | X) = 3/5 = 0.6000
P(Adulterated | Y) = 1/3 = 0.3333
P(Adulterated) overall = 19/45 = 0.4222

P(Shop X | Adulterated) = 9/19 = 0.4737
P(Shop Y | Adulterated) = 10/19 = 0.5263
```

12. A warehouse has two sections. Section A contains 15 defective items and 35 non-defective items, while Section B contains 25 defective items and 25 non-defective items. One section is chosen at random, but Section A is three times as likely to be chosen as Section B. An item selected from the chosen section is found to be defective.

CODE :

```
from fractions import Fraction
# Prior probabilities: A is three times as likely to be chosen as B
P_A = Fraction(3, 4)
P_B = Fraction(1, 4)
# Likelihoods: P(defective | section)
P_D_given_A = Fraction(15, 50)
P_D_given_B = Fraction(25, 50)
# Total probability of picking a defective item (Law of Total Probability)
P_D = P_A * P_D_given_A + P_B * P_D_given_B
# Posterior probabilities via Bayes' theorem
P_A_given_D = (P_A * P_D_given_A) / P_D
P_B_given_D = (P_B * P_D_given_B) / P_D
print(f"P(Section A chosen) = {P_A} = {float(P_A):.4f}")
print(f"P(Section B chosen) = {P_B} = {float(P_B):.4f}")
print(f"P(Defective | A) = {P_D_given_A} = {float(P_D_given_A):.4f}")
print(f"P(Defective | B) = {P_D_given_B} = {float(P_D_given_B):.4f}")
print(f"P(Defective) overall = {P_D} = {float(P_D):.4f}")
print()
print(f"P(Section A | Defective) = {P_A_given_D} = {float(P_A_given_D):.4f}")
print(f"P(Section B | Defective) = {P_B_given_D} = {float(P_B_given_D):.4f}")
```

OUTPUT :



13. In a binary classification problem using a deep learning model with a K-Nearest Neighbor classifier, the model produces True Positive (TP) = 120 and True Negative (TN) = 150.The dataset contains 1000 samples per class, and 20% of the data is used for testing.
Find the following: Accuracy, Precision, Sensitivity (Recall), Specificity.

CODE :

```
# Dataset info
total_samples = 1000 * 2
test_fraction = 0.20
test_samples = int(total_samples * test_fraction)
samples_per_class_test = test_samples // 2
# Given values
TP = 120
TN = 150
actual_positives = samples_per_class_test
actual_negatives = samples_per_class_test
# Derive FN and FP
FN = actual_positives - TP
FP = actual_negatives - TN
print(f"Test samples: {test_samples} (Positive: {actual_positives}, Negative:
{actual_negatives})")
print(f"TP = {TP}, TN = {TN}, FP = {FP}, FN = {FN}")
print()
# Metrics
accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
sensitivity = TP / (TP + FN)
specificity = TN / (TN + FP)
print(f"Accuracy    = (TP+TN)/(TP+TN+FP+FN) = {accuracy:.4f}
({accuracy*100:.2f}%)")
print(f"Precision   = TP/(TP+FP)            = {precision:.4f}
({precision*100:.2f}%)")
print(f"Sensitivity = TP/(TP+FN)            = {sensitivity:.4f}
({sensitivity*100:.2f}%)")
print(f"Specificity = TN/(TN+FP)            = {specificity:.4f}
({specificity*100:.2f}%)")
```

OUTPUT :

```
total_samples = 1000 * 2
test_fraction = 0.20
test_samples = int(total_samples * test_fraction)
samples_per_class_test = test_samples // 2
TP = 120
TN = 150
actual_positives = samples_per_class_test
actual_negatives = samples_per_class_test
FN = actual_positives - TP
FP = actual_negatives - TN
print(f"Test samples: {test_samples} (Positive: {actual_positives}, Negative: {actual_negatives})")
print(f"TP = {TP}, TN = {TN}, FP = {FP}, FN = {FN}")
print()
# Metrics
accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
sensitivity = TP / (TP + FN)
specificity = TN / (TN + FP)
print(f"Accuracy    = (TP+TN)/(TP+TN+FP+FN) = {accuracy:.4f} ({accuracy*100:.2f}%)")
print(f"Precision   = TP/(TP+FP)            = {precision:.4f} ({precision*100:.2f}%)")
print(f"Sensitivity = TP/(TP+FN)            = {sensitivity:.4f} ({sensitivity*100:.2f}%)")
print(f"Specificity = TN/(TN+FP)            = {specificity:.4f} ({specificity*100:.2f}%)")
```

```
Test samples: 400 (Positive: 200, Negative: 200)
TP = 120, TN = 150, FP = 50, FN = 80

Accuracy    = (TP+TN)/(TP+TN+FP+FN) = 0.6750 (67.50%)
Precision   = TP/(TP+FP)            = 0.7059 (70.59%)
Sensitivity = TP/(TP+FN)            = 0.6000 (60.00%)
Specificity = TN/(TN+FP)            = 0.7500 (75.00%)
```

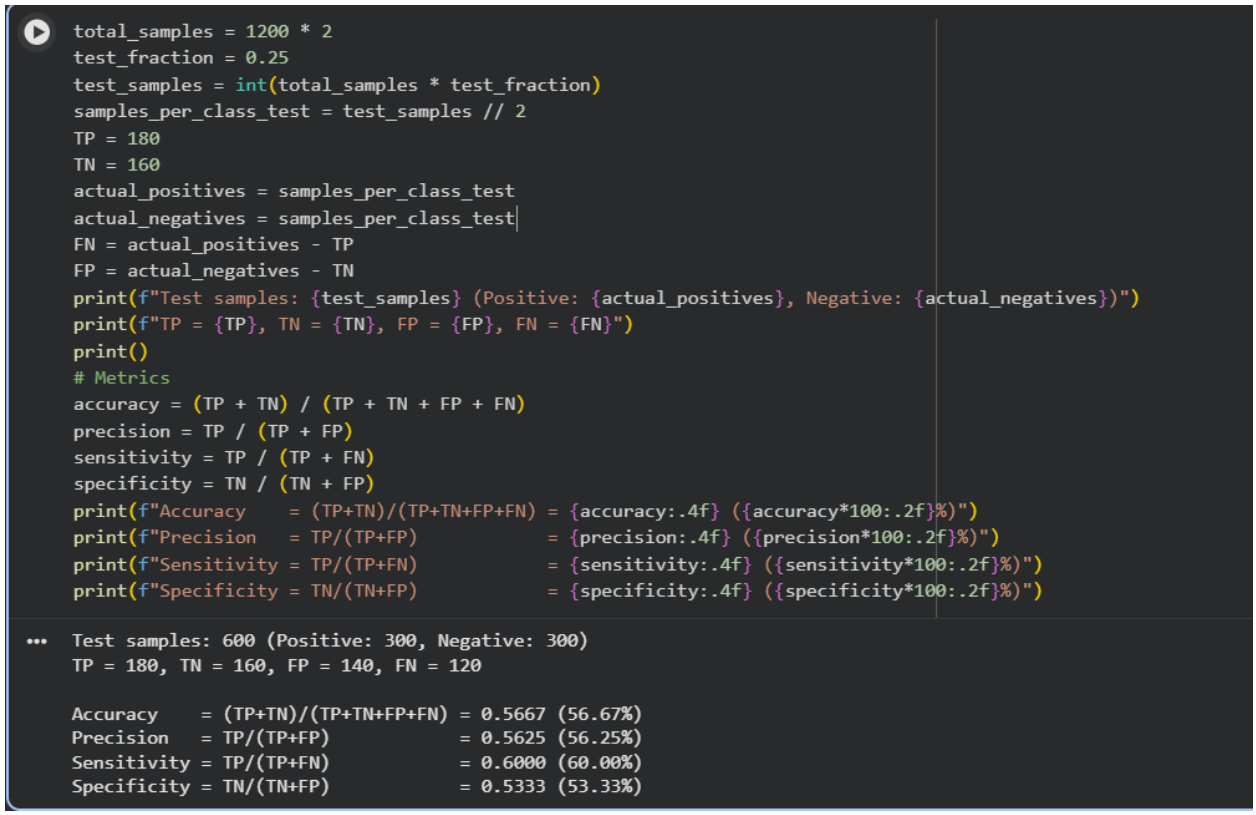
14. A medical image classification system using a deep learning framework combined with K-Nearest Neighbor (KNN) yields TP = 180 and TN = 160. Each class contains 1200 samples, and 25% of the dataset is used as test data. Calculate: Accuracy Precision Sensitivity Specificity.

CODE :

```
# Dataset info
total_samples = 1200 * 2
test_fraction = 0.25
test_samples = int(total_samples * test_fraction)
samples_per_class_test = test_samples // 2
# Given values
TP = 180
TN = 160
# Actual positives = 300, Actual negatives = 300
actual_positives = samples_per_class_test
actual_negatives = samples_per_class_test
# Derive FN and FP
FN = actual_positives - TP
```

```
FP = actual_negatives - TN
print(f"Test samples: {test_samples} (Positive: {actual_positives}, Negative:
{actual_negatives})")
print(f"TP = {TP}, TN = {TN}, FP = {FP}, FN = {FN}")
print()
# Metrics
accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
sensitivity = TP / (TP + FN)
specificity = TN / (TN + FP)
print(f"Accuracy    = (TP+TN)/(TP+TN+FP+FN) = {accuracy:.4f}
({accuracy*100:.2f}%)")
print(f"Precision   = TP/(TP+FP)            = {precision:.4f}
({precision*100:.2f}%)")
print(f"Sensitivity = TP/(TP+FN)            = {sensitivity:.4f}
({sensitivity*100:.2f}%)")
print(f"Specificity = TN/(TN+FP)            = {specificity:.4f}
({specificity*100:.2f}%)")
```

OUTPUT :



15. A deep learning model is trained for a classification task, and the following loss values are observed : (5 Marks)

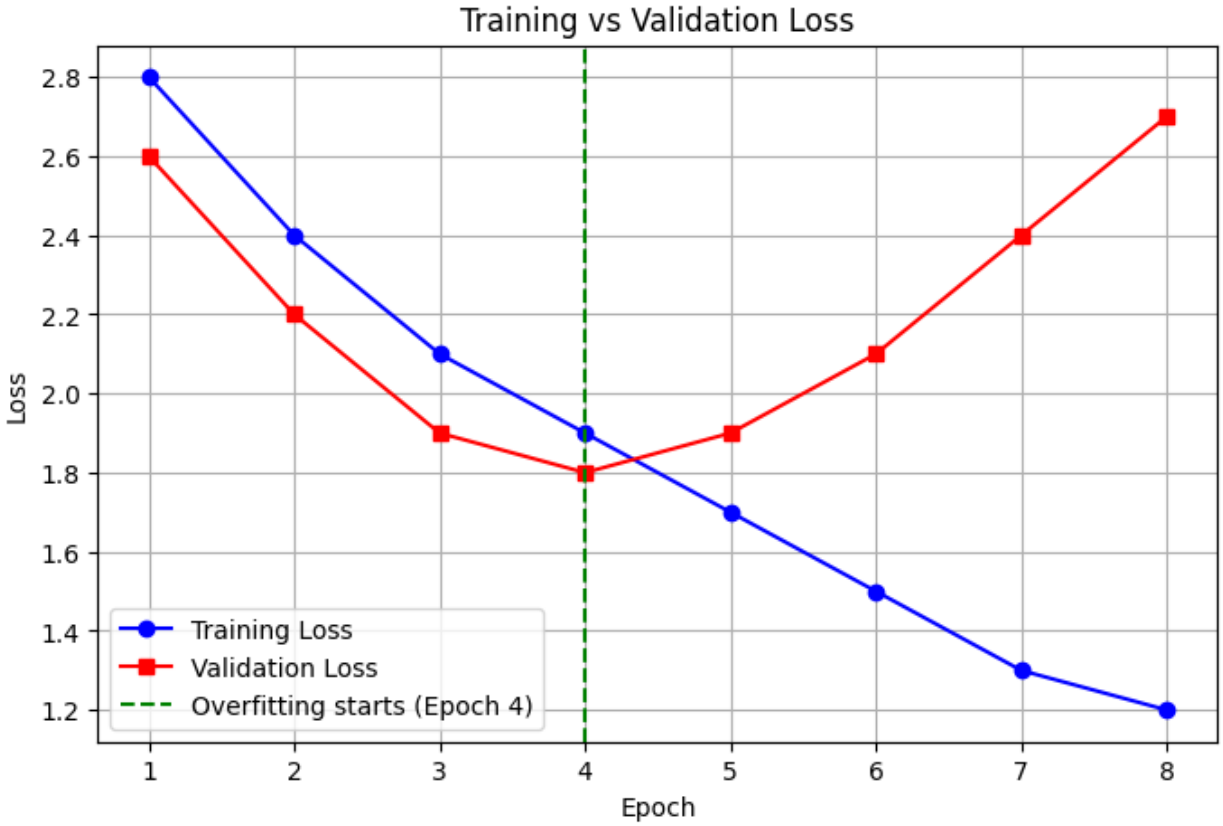
i. Epoch 1: Training Loss = 2.8, Validation Loss = 2.6
ii. Epoch 2: Training Loss = 2.4, Validation Loss = 2.2
iii. Epoch 3: Training Loss = 2.1, Validation Loss = 1.9
iv. Epoch 4: Training Loss = 1.9, Validation Loss = 1.8
v. Epoch 5: Training Loss = 1.7, Validation Loss = 1.9
vi. Epoch 6: Training Loss = 1.5, Validation Loss = 2.1
vii. Epoch 7: Training Loss = 1.3, Validation Loss = 2.4
viii. Epoch 8: Training Loss = 1.2, Validation Loss = 2.7

CODE :

```
import matplotlib.pyplot as plt
# Given data
epochs = [1, 2, 3, 4, 5, 6, 7, 8]
train_loss = [2.8, 2.4, 2.1, 1.9, 1.7, 1.5, 1.3, 1.2]
val_loss   = [2.6, 2.2, 1.9, 1.8, 1.9, 2.1, 2.4, 2.7]
# Plot the loss curves
plt.figure(figsize=(8, 5))
plt.plot(epochs, train_loss, marker='o', label='Training Loss',
color='blue')
plt.plot(epochs, val_loss, marker='s', label='Validation Loss', color='red')
plt.axvline(x=4, color='green', linestyle='--', label='Overfitting starts
(Epoch 4)')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training vs Validation Loss')
plt.legend()
plt.grid(True)
plt.show()
# Analysis
min_val_loss = min(val_loss)
best_epoch = epochs[val_loss.index(min_val_loss)]
print(f"Minimum Validation Loss = {min_val_loss} at Epoch {best_epoch}")
print()
print("Observations:")
print("- From Epoch 1 to 4: Both training loss and validation loss decrease
together.")
print("  -> The model is learning and generalizing well (underfitting
reduces).")
```

```
print(f"- Epoch {best_epoch}: Validation loss reaches its minimum
({min_val_loss}).")
print("  -> This is the optimal stopping point (best generalization).")
print("- From Epoch 5 to 8: Training loss keeps decreasing (2.8 -> 1.2),")
print("  but validation loss keeps increasing (1.8 -> 2.7).")
print("  -> This gap indicates OVERFITTING: the model is memorizing
training")
print("     data instead of generalizing to unseen data.")
```

OUTPUT :



Minimum Validation Loss = 1.8 at Epoch 4

Observations:

- From Epoch 1 to 4: Both training loss and validation loss decrease together.
  -> The model is learning and generalizing well (underfitting reduces).
- Epoch 4: Validation loss reaches its minimum (1.8).
  -> This is the optimal stopping point (best generalization).
- From Epoch 5 to 8: Training loss keeps decreasing (2.8 -> 1.2),
  but validation loss keeps increasing (1.8 -> 2.7).
  -> This gap indicates OVERFITTING: the model is memorizing training
  data instead of generalizing to unseen data.

```python
import matplotlib.pyplot as plt
# Given data
epochs = [1, 2, 3, 4, 5, 6, 7, 8]
train_loss = [2.8, 2.4, 2.1, 1.9, 1.7, 1.5, 1.3, 1.2]
val_loss   = [2.6, 2.2, 1.9, 1.8, 1.9, 2.1, 2.4, 2.7]
# Plot the loss curves
plt.figure(figsize=(8, 5))
plt.plot(epochs, train_loss, marker='o', label='Training Loss', color='blue')
plt.plot(epochs, val_loss, marker='s', label='Validation Loss', color='red')
plt.axvline(x=4, color='green', linestyle='--', label='Overfitting starts (Epoch 4)')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training vs Validation Loss')
plt.legend()
plt.grid(True)
plt.show()
# Analysis
min_val_loss = min(val_loss)
best_epoch = epochs[val_loss.index(min_val_loss)]
print(f"Minimum Validation Loss = {min_val_loss} at Epoch {best_epoch}")
print()
print("Observations:")
print("- From Epoch 1 to 4: Both training loss and validation loss decrease together.")
print("  -> The model is learning and generalizing well (underfitting reduces).")
print(f"- Epoch {best_epoch}: Validation loss reaches its minimum ({min_val_loss}).")
print("  -> This is the optimal stopping point (best generalization).")
print("- From Epoch 5 to 8: Training loss keeps decreasing (2.8 -> 1.2),")
print("  but validation loss keeps increasing (1.8 -> 2.7).")
print("  -> This gap indicates OVERFITTING: the model is memorizing training")
print("     data instead of generalizing to unseen data.")
```


```
Minimum Validation Loss = 1.8 at Epoch 4

Observations:
- From Epoch 1 to 4: Both training loss and validation loss decrease together.
  -> The model is learning and generalizing well (underfitting reduces).
- Epoch 4: Validation loss reaches its minimum (1.8).
  -> This is the optimal stopping point (best generalization).
- From Epoch 5 to 8: Training loss keeps decreasing (2.8 -> 1.2),
```